

CHECKPOINT 5 — Bug Hunt PetFiap

Disciplina: Programação Orientada a Objetos

Projeto: PetFiap — pet shop & clínica veterinária

Valor: 10,0 pontos

Conteúdo: Aulas 1 a 15 (POO + Exceções + JDBC/DAO + Spring Boot + Padrões de projeto + Testes unitários)

Entrega: em grupo — repositório público no GitHub (link no Teams)

O Problema Real

Você acaba de entrar na equipe do PetFiap, o pet shop da FIAP. O estagiário anterior entregou o sistema de agendamentos e pediu demissão, dizendo que "ia abrir um refúgio de animais numa praia desertaaaaa".

Ele jurou de pés juntos que "o código funciona". E funciona mesmo: o projeto compila e a API sobe sem nenhum erro. O problema é o que acontece depois...

Desta vez você não está sozinho na caça: junto com o código legado, o estagiário deixou (milagre!) uma suíte de testes unitários, escrita com JUnit 5 + Mockito, igual à Aula 15. A suíte descreve o contrato oficial do sistema. Só que hoje ela está vermelha: o código não cumpre o que os testes exigem.

Sua missão: deixar a suíte 100% verde corrigindo os bugs, escrever os testes que faltam, adequar o código às boas práticas de Clean Code e documentar tudo, exatamente como um dev sênior faria ao assumir um código legado.

O que você recebeu

O projeto petfiap-bughunt é uma API Spring Boot com:

Model (POO + Aula 14): Atendimento (abstrata) → Banho, Tosa, ConsultaVeterinaria. As regras de negócio moram no model: preço por porte do pet, pontos de fidelidade e duração, além das transições de status (concluir()/cancelar()).

Padrões de projeto (Aula 14):

GeradorProtocolo (Singleton — numeração sequencial dos protocolos)

AtendimentoFactory (Factory — o único lugar que conhece as subclasses concretas)

AtendimentoBuilder (Builder — monta o atendimento passo a passo no controller).

Serviço: AgendaService — regras de agendamento (conflito de horário do pet), conclusão e cancelamento.

Persistência: Spring Data JPA (Aten

dimentoRepository) — tabela única atendimentos (como na Aula 13)

API (controller, estilo Aulas 13/14):

AtendimentoController — com ResponseEntity e status HTTP adequados

Exceções customizadas (unchecked, Aula 11): AtendimentoNaoEncontradoException

HorarioOcupadoException

StatusInvalidoException

Suíte de testes unitários (Aula 15) em src/test/java — 20 testes JUnit 5 + Mockito sobre o model, os padrões e o serviço (sem banco, sem Spring). 9 deles estão vermelhos hoje. A suíte está correta: quando ela falha, o problema está no código de produção.

Banco Oracle FIAP (como nas Aulas 12/13) — o grupo troca SEU_RM e SUA_SENHA no application.properties pelas próprias credenciais apenas para rodar a API (os testes unitários não usam banco)

O Contrato (o código DEVE cumprir isto)

Parte A — O contrato com a suíte de testes (o inegociável)

O estado de entrega é: suíte vermelha (20 testes, 9 falhando). O estado final obrigatório é: suíte 100% verde (20 testes, 0 falhas) e os testes novos da Parte B passando.

A suíte é a fonte de verdade: cada teste vermelho é um sintoma apontando para um bug. Corrija a causa no código de produção, nunca "conserte" o teste para ele passar (a suíte entregue já está certa; alterá-la anula o item, conforme as regras da seção 4).

Parte B — As regras sem cobertura (6 testes faltantes)

Existem 6 regras de negócio do contrato que a suíte entregue NÃO cobre, ninguém escreveu teste para elas. Faz parte da missão:

Descobrir quais regras estão sem cobertura (lendo o contrato das tabelas abaixo e comparando com os testes existentes);

Escrever um teste novo para cada regra descoberta, no padrão da suíte (AAA, nome deve...Quando..., mock onde houver dependência);

Se o teste novo ficar vermelho, você achou um bug escondido: conserte a causa e deixe-o verde. Se ficar verde de cara, a regra já estava correta, o teste passa a proteger você de regressões futuras.

Dica: 4 das 6 regras sem cobertura escondem bugs; 2 já estão corretas. Descobrir qual é qual faz parte do jogo.

Parte C — As regras de negócio (o que os testes exigem)

Preço, pontos e duração por tipo de atendimento:

Tipo	PEQUENO	MEDIO	GRANDE	Pontos	Duração
BANHO	R\$ 60,00	R\$ 80,00	R\$ 100,00	20	45 min
TOSA	R\$ 70,00	R\$ 90,00	R\$ 120,00	30	60 min

Tipo	PEQUENO	MEDIO	GRANDE	Pontos	Duração
CONSULTA	R\$ 150,00 fixo (porte não muda o preço)	150,00	150,00	50	30 min

Regras de agendamento:

Regra	Comportamento esperado
Agendar atendimento válido	Atendimento criado com status AGENDADO e protocolo sequencial (1, 2, 3...)
Agendar para o mesmo pet, mesmo horário, com atendimento AGENDADO	Recusar com <code>HorarioOcupadoException</code> — nada é salvo
Agendar com data/hora no passado	Recusar com <code>IllegalArgumentExpection</code> — o banco nem é consultado
Nome do pet ou porte ausentes no Builder	Recusar no <code>construir()</code> — o objeto só nasce válido
Tipo inexistente (ex.: "VACINA")	Recusar com mensagem clara

Regras de status:

Operação	AGENDADO	CONCLUIDO	CANCELADO
concluir()	✅ vira CONCLUIDO	❌ recusa (<code>StatusInvalidoException</code>)	❌ recusa
cancelar()	✅ vira CANCELADO	❌ recusa (atendimento já realizado)	❌ recusa

Regras de consulta:

Regra	Comportamento esperado
Buscar por id inexistente	Lança <code>AtendimentoNaoEncontradoException</code> — nunca retorna null "como se nada tivesse acontecido"
Protocolos	Gerados pelo Singleton <code>GeradorProtocolo</code> : uma única instância, numeração global sequencial

A caça

Estão escondidos no projeto:

🐛 12 bugs de comportamento (o código faz algo errado ou aceita o que não deve)

🔧 6 violações de Clean Code (o código funciona, mas fere boas práticas vistas em aula)

📏 6 regras sem cobertura (testes que deveriam existir e não existem)

Nem todo bug aparece nos 9 testes vermelhos: alguns só aparecem quando você escreve o teste faltante e um deles só aparece lendo o código com atenção, como num code review de verdade.

Regras do jogo

1. NÃO reescreva o projeto do zero. Corrija o que existe, com mudanças mínimas.
2. NÃO altere os 20 testes entregues. A suíte é o contrato — se um teste vermelho "aprontar", o caminho é consertar o código, não o teste. Alterar teste entregue para passar anula o item.
3. Um commit por correção, com mensagem semântica:

```
fix: bug01 - corrige singleton que criava nova instancia a cada getInstancia
test: teste03 - cobre cancelamento de atendimento concluido
refactor: clean01 - renomeia parametros de uma letra na factory
```

(numere os bugs na ordem em que você os encontrou)

4. NÃO adicione bibliotecas/frameworks além dos que já estão no pom.xml (JUnit 5 e Mockito já vêm no spring-boot-starter-test).
5. NÃO force o contrato: correção com valores hardcoded para "passar" nos testes (ex.: if (petNome.equals("Rex"))) anula o item.
6. Registre cada achado no README (template AVALIACAO_README_TEMPLATE.md): sintoma → causa raiz → correção → conceito da disciplina envolvido.

Como rodar e testar

Este roteiro assume que você participou das Aulas 11 a 15. Requisito: JDK 17 ou superior (o mesmo usado nas aulas). Recebido o zip do Teams, descompacte na sua pasta de projetos:

```
unzip petfiap-bughunt.zip -d ~/projetos/
cd ~/projetos/petfiap-bughunt
```

1. No Eclipse: File → Import → Maven → Existing Maven Projects e selecione a pasta do projeto
2. A suíte (o seu termômetro): clique com o botão direito na pasta src/test/java → Run As → JUnit Test. A aba JUnit abre com a barra vermelha  e os 9 testes falhando listados. Cada teste vermelho é um sintoma: clique nele para ver a mensagem (ex.: expected: <Rex> but was: <null>)
3. A API (opcional para entender o sistema): troque SEU_RM e SUA_SENHA no src/main/resources/application.properties pelas credenciais do grupo e rode a classe PetFiapApplication (Run , igualzinho na Aula 13). Quando aparecer Started PetFiapApplication, a API está em http://localhost:8080. Essa alteração fica só na sua máquina: o repositório deve continuar com SEU_RM/SUA_SENHA
4. Testes com curl (se subir a API):

```
curl -i -X POST "http://localhost:8080/api/atendimentos?tipo=BANHO&petNome=Rex&porte=P
EQUENO&tutorNome=Ana&dataHora=2026-12-01T10:00"
curl http://localhost:8080/api/atendimentos/1/resumo
```

💡 A suíte roda SEM banco e SEM rede — em qualquer máquina, em segundos. Se o Oracle da FIAP estiver fora do ar, você ainda consegue trabalhar: o @Mock do Mockito substitui o repository. (Foi para isso que a Aula 15 existe.)

Roteiro sugerido: rode a suíte e leia os 9 vermelhos — cada um aponta um sintoma. Para cada divergência com o contrato (seção 3), anote: o que fiz → o que veio → o que o contrato dizia que deveria vir. Depois vasculhe as tabelas da Parte C procurando as regras sem teste e escreva os 6 testes novos.

O ciclo da caça (repita até a suíte ficar 100% verde)

1. **Rode** a suíte (**Run As** → **JUnit Test**) e escolha um teste vermelho
2. **LEIA** a mensagem de falha: o que era esperado e o que veio?
3. **LEIA** o código envolvido (model → builder/**factory** → service)
4. **Ache** a causa raiz — não conserte o sintoma, conserte a causa
5. **Corrija** com a menor mudança possível (e **NÃO** mexa no teste)
6. **Rode** a suíte de novo: aquele teste ficou verde? Alguém quebrou?
7. **Commit:**

```
git add .  
git commit -m "fix: bugNN - <o que estava errado e o que voce fez>"
```

Dicas de caça:

- Passe olho pela suíte inteiro antes de começar. Os testes são curtos e legíveis — eles são a especificação do sistema em forma de código.
- As regras de negócio moram no model: preço, pontos, duração e transições de status (Atendimento.concluir/cancelar). Os padrões da Aula 14 (Singleton/Factory/Builder) montam e criam o atendimento — desconfie deles também.
- Compare subclasses entre si: se Banho, Tosa e ConsultaVeterinaria deveriam seguir o mesmo padrão e uma delas destoa, desconfie. Um método que "quase" sobrescreve (mudou a assinatura?) é um clássico.
- Procure por catch vazio/genérico, == entre objetos, this ausente, instância não guardada no singleton, case trocado na factory e exceções que são lançadas mas nunca chegam a quem chamou.
- Bugs em cascata são normais: alguns sintomas só aparecem depois de corrigir quem os bloqueava. Rode a suíte depois de CADA correção.

Entrega (em grupo)

1. O grupo cria um repositório público no GitHub: cp5-bughunt-<nome-do-grupo> (ex.: cp5-bughunt-debugadores)
2. Um integrante sobe o projeto completo já com o primeiro commit do estado original (com o application.properties intacto, SEU_RM/SUA_SENHA):

```
git init  
git add .  
git commit -m "chore: estado original do projeto recebido"  
git remote add origin https://github.com/seugrupo/cp5-bughunt-nome-do-grupo.git  
git push -u origin main
```

3. Façam as correções com um commit por bug, um commit por item de Clean Code e um commit por teste novo (podem dividir entre os integrantes — cada correção continua sendo um commit). Só código de correção nos commits: nunca o application.properties com o RM/senha reais do grupo
4. Copie o AVALIACAO_README_TEMPLATE.md para a raiz como README.md e preencha tudo, incluindo a tabela com todos os integrantes do grupo

5. git push e entregue o link no Teams — o mesmo link para todo o grupo

Penalidades

Situação	Penalidade
Repositório privado	-2,0
Commit único gigante com todas as correções	-1,5
README incompleto ou em branco	-1,5 por seção
Projeto não compila / suíte não roda	-3,0
Alterar teste entregue para fazê-lo passar	anula o item do bug correspondente
Commitar o RM/senha reais do grupo no repositório	-0,5 (e trocar a senha em seguida)
Hardcode/jeitinho para enganar a suíte	nota zero

Troubleshooting

Problema	Causa provável	Solução
Projeto com erros de import no Eclipse	não foi importado como Maven	File → Import → Maven → Existing Maven Projects; se persistir, clique com o botão direito no projeto → Maven → Update Project
Não acho "Run As → JUnit Test"	seleção errada	clique com o botão direito na pasta src/test/java (ou numa classe de teste) → Run As → JUnit Test
Barra vermelha com "Unable to find matcher" ou erro de stubbing	teste escrito com when para um mock que o código não chama nesse caminho	confira se o cenário do Arrange bate com o Act — o mock é literal, só faz o que você ensinou
UnsupportedClassVersionError	Eclipse rodando com JDK antigo (menor que 17)	Window → Preferences → Java → Installed JREs → selecione o JDK 17 usado em aula
Port 8080 was already in use	a aplicação anterior ainda está rodando	pare com o botão vermelho do Console e rode de novo
ORA-01017: invalid username/password	credenciais não configuradas	troque SEU_RM e SUA_SENHA no application.properties (só para RODAR a API — testes não precisam)
Erro de conexão/timeout no Oracle	rede indisponível	os testes unitários rodam mesmo assim — a caça não para
Teste que passava voltou a falhar	regressão (sua correção quebrou outra regra)	normal: rode a suíte inteira depois de cada correção
ORA-00955: name is already used ou erro de coluna/tabela	tabela antiga no schema do grupo, com estrutura diferente	apague a tabela antiga no seu schema — o Hibernate recria na próxima execução

Checklist final antes de entregar

- Suíte entregue 100% verde: 20 testes, 0 falhas (src/test/java → Run As → JUnit Test)
- 6 testes novos escritos, cada um com commit próprio (test: ...)
- Nenhum dos 20 testes entregues foi alterado (só código de produção mudou)
- O projeto compila e a API sobe pelo play no Eclipse (opcional, mas recomendado)
- application.properties no repositório continua com SEU_RM/SUA_SENHA
- Primeiro commit = estado original; depois, 1 commit por correção
- README preenchido (integrantes, bugs, Clean Code, testes novos, 6 reflexões)
- Repositório público e link entregue no Teams (o mesmo para todo o grupo)

O que está sendo avaliado

Critério	Pontos
Bugs corrigidos (12 × 0,5)	6,0
Ajustes de Clean Code (6 × 0,25)	1,5
Testes novos para regras sem cobertura (6 × 0,25)	1,5
README: documentação dos achados + perguntas de reflexão	1,0
Total	10,0

Boa caça! 🏹